



Transformer-Enhanced Context Retrieval using Modern Hopfield Networks

Practical Work Report

Student: Tamas Scheidl

Program: Artificial Intelligence Master

University: Johannes Kepler University Linz

Date: April 12, 2026

1 Introduction

Context-aware retrieval and representation learning have become increasingly important in modern deep learning applications, particularly in tasks such as few-shot learning, similarity learning, and recommendation systems. Traditional neural network architectures often struggle to effectively incorporate contextual information when only limited data is available. Memory-based approaches, such as Modern Hopfield Networks, provide a promising solution by enabling associative retrieval of relevant patterns from stored representations. These architectures allow models to dynamically retrieve contextual information and refine embeddings based on similarity relationships [1].

The MHNfs framework provides a modular architecture for context-based retrieval using Modern Hopfield Networks and has demonstrated strong performance in few-shot learning scenarios [2]. However, the original implementation primarily relies on fixed retrieval mechanisms and does not fully leverage recent advances in transformer-based attention models. Transformer architectures have become a dominant approach for learning contextual relationships due to their flexible attention mechanisms and ability to capture complex dependencies between embeddings [3]. Integrating these mechanisms into the MHNfs framework offers the potential to improve context retrieval and representation learning.

In this work, an improved architecture is proposed that enhances the original MHNfs framework by incorporating transformer-based components. The proposed model introduces a Cross-Attention Module that enables interaction between query embeddings and support embeddings. This allows the model to dynamically retrieve relevant contextual information using multi-head attention. In addition, a Context Module based on Modern Hopfield Networks is used to iteratively refine embeddings through associative memory retrieval. Finally, a Similarity Module aggregates similarity scores and produces final predictions.

The proposed architecture focuses on improving flexibility, stability, and modularity. Multi-head attention enables the model to capture relationships across different representation subspaces, while residual connections improve stability during iterative retrieval. Furthermore, the Similarity Module implements multiple aggregation strategies, including sum, softmax, and log-sum-exp, to evaluate different similarity behaviors.

The objective of this work is to analyze whether transformer-enhanced context retrieval can improve representation learning within the Modern Hopfield framework. The modular design allows independent evaluation of each component and supports future extensions. Overall, the proposed transformer-enhanced architecture provides a flexible and extensible framework for context-aware retrieval tasks and contributes to the development of improved Modern Hopfield-based learning systems.

2 Architecture Overview

The proposed architecture improves the original MHNfs framework by incorporating transformer-based components and enhancing context retrieval using Modern Hopfield Networks. The model follows a modular design consisting of three main components: the Cross-Attention Module, the Context Module, and the Similarity Module. These components work together to enhance

representation learning and context-based retrieval.

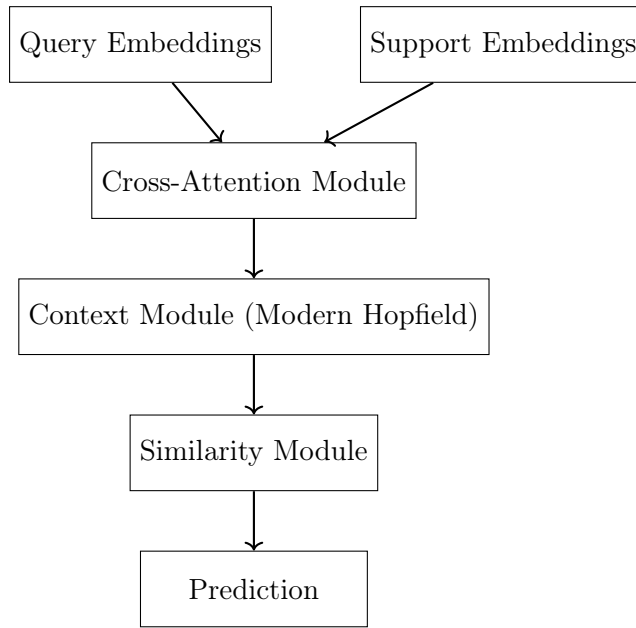


Figure 1: Architecture of the proposed transformer-enhanced Modern Hopfield model

The overall pipeline begins with embedding representations of query samples and support set samples. The Cross-Attention Module first processes these embeddings to enable interaction between query and support representations. The output is then refined using the Context Module, which applies Modern Hopfield-based associative memory retrieval. Finally, the Similarity Module aggregates the refined embeddings and produces the final prediction.

Component	Purpose	Important Features
Cross-Attention Module	Query-support interaction	Multi-head attention, residual connections, gating mechanism
Context Module	Embedding refinement	Modern Hopfield retrieval, multi-head retrieval, iterative updates
Similarity Module	Prediction generation	Sum, softmax, log-sum-exp aggregation strategies

Table 1: Components of the transformer-enhanced Modern Hopfield architecture

2.1 Cross-Attention Module

The Cross-Attention Module introduces transformer-style cross-attention into the architecture, allowing query embeddings to interact directly with support set embeddings. Unlike self-attention, where queries, keys, and values originate from the same input, cross-attention enables the model to retrieve relevant contextual information from external support samples. This mechanism is particularly beneficial for few-shot learning, where contextual information from limited support examples is critical.

The implemented module follows a multi-head scaled dot-product attention mechanism, where query embeddings attend to both active and inactive support embeddings. In addition,

separate bias embeddings are introduced to distinguish between active and inactive samples, improving the model’s ability to differentiate molecular activity classes. The outputs are further stabilized using normalization layers and residual connections.

Compared to the original MHNfs implementation, the Cross-Attention module was redesigned to improve stability, efficiency, and expressiveness. The original attention mechanism is illustrated in Figure 2, where query, key, and value embeddings originate from the same representation. In contrast, the proposed cross-attention mechanism (Figure 3) enables direct interaction between query and support embeddings.

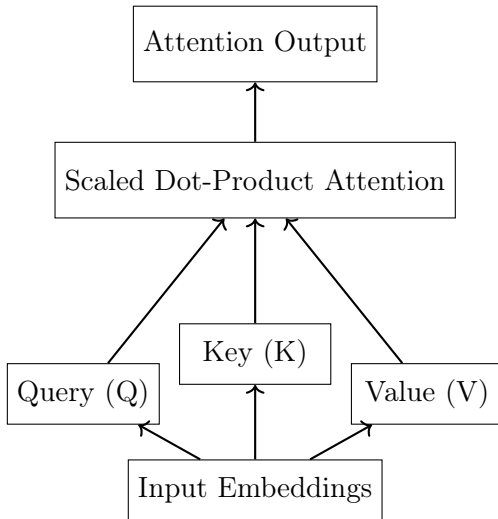


Figure 2: Original attention mechanism where query, key, and value originate from the same input

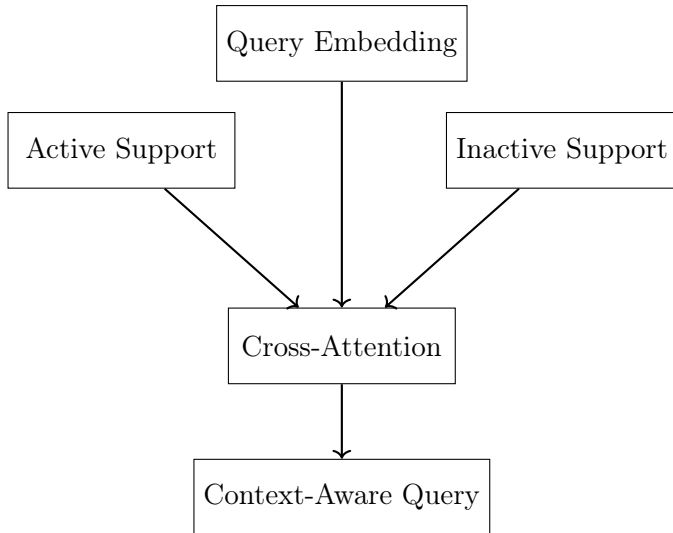


Figure 3: Proposed cross-attention mechanism between query and support embeddings

In addition to cross-attention, several improvements were introduced to enhance model stability and performance:

- **True Cross-Attention ($Q \neq K, V$)** The query explicitly attends to support embeddings, improving retrieval quality.

- **Scaled Dot-Product Attention** The implementation uses PyTorch’s optimized attention function for improved numerical stability and efficiency.
- **Active / Inactive Bias Embeddings** Separate bias embeddings help distinguish between molecule activity classes.
- **Gated Residual Connections** Learnable gates control how much new information is incorporated.
- **Mixture-of-Experts (MoE)** Multiple expert networks improve representation learning and model capacity.
- **RMS Normalization** Normalization improves training stability.
- **Residual Scaling** Prevents unstable updates and improves gradient flow.
- **Attention Temperature Scaling** Learnable temperature improves attention sharpness and flexibility.

These enhancements result in a more stable and expressive cross-attention mechanism that improves context retrieval and representation learning. The redesigned module is particularly suited for few-shot molecular classification tasks, where efficient contextual reasoning is essential.

2.2 Context Module

The Context Module is based on Modern Hopfield Networks and performs associative memory retrieval to refine query and support embeddings. Unlike traditional attention mechanisms that perform a single update, the proposed module applies iterative retrieval steps, allowing the model to progressively refine representations. This iterative process improves contextual reasoning and stabilizes representation learning.

The Context Module receives query embeddings together with active and inactive support embeddings. First, the embeddings are normalized and projected into a shared representation space. Then, a top-k selection mechanism identifies the most relevant context samples. This selection reduces noise and focuses retrieval on the most informative support examples.

The refined embeddings are then processed using a Modern Hopfield layer. The Hopfield module performs multi-head associative retrieval using query, key, and value projections, similar to transformer attention. However, unlike transformers, the Hopfield layer follows an energy-based formulation and supports iterative retrieval. The retrieval output is combined with the original embeddings using gated residual updates to improve stability.

After each retrieval step, a transformer-style feed-forward network (FFN) further refines the embeddings. This multi-step refinement process continues for a fixed number of iterations. The overall workflow of the proposed Context Module is illustrated in Figure 4.

The core component of this module is the Modern Hopfield layer, which performs associative memory retrieval. Unlike classical Hopfield networks, the implemented version follows a transformer-like architecture with multi-head attention and learnable temperature scaling. This allows the model to retrieve patterns more effectively from high-dimensional embeddings. The structure of the Modern Hopfield retrieval is illustrated in Figure 5.

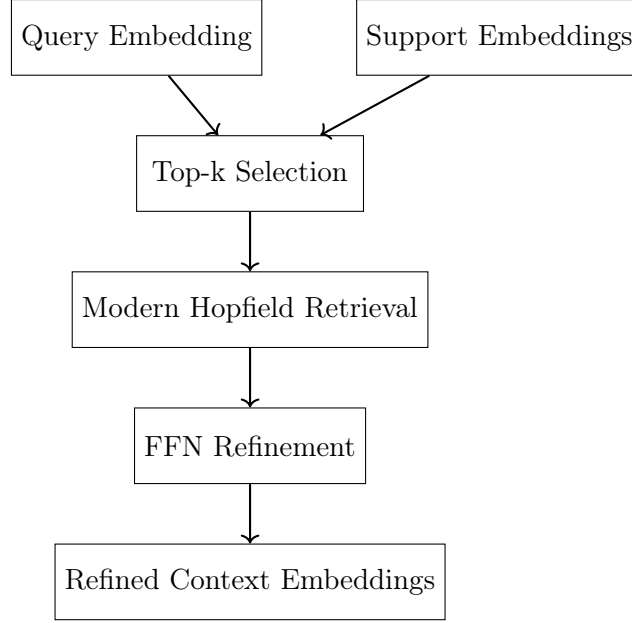


Figure 4: Proposed Context Module using iterative Modern Hopfield retrieval

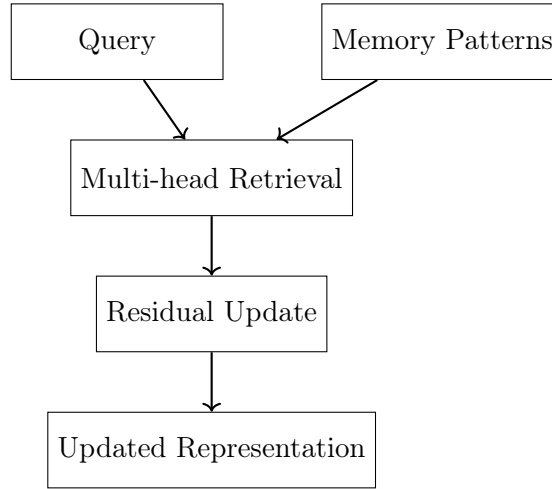


Figure 5: Modern Hopfield associative retrieval mechanism

Compared to the original implementation, several improvements were introduced:

- **Multi-step retrieval** Iterative retrieval refines representations progressively instead of a single update.
- **Top-k memory selection** Only the most relevant context embeddings are used for retrieval.
- **Learnable gating** Controls how much retrieved information is incorporated.
- **Separate projections** Different projections for query, active, and inactive embeddings improve specialization.
- **Layer normalization and L2 normalization** Improves numerical stability and representation consistency.

- **Transformer-style FFN refinement** Enhances representation quality after each retrieval step.
- **Multi-head Hopfield retrieval** Captures different similarity patterns simultaneously.
- **Learnable temperature (β)** Controls retrieval sharpness adaptively.

These improvements result in a more stable and expressive context retrieval mechanism. The proposed Context Module enables iterative refinement of embeddings, improving representation learning and retrieval performance for few-shot molecular classification tasks.

2.3 Similarity Module

The Similarity Module computes similarity-based scores between a query embedding and a set of support set embeddings. It extends a standard dot-product similarity formulation into a more flexible and robust multi-head architecture designed for few-shot learning and associative matching.

Given a query embedding and a support set, the module first optionally applies learnable linear projections followed by an optional L2 normalization step. The representations are then split into multiple attention heads to enable parallel similarity computation across different subspaces of the embedding space.

For each head, scaled dot-product similarities are computed between the query and all support elements. These similarities are further stabilized using temperature scaling and optionally masked to ignore padded support elements. The module also introduces separate weighting for positive and negative similarity contributions, allowing finer control over similarity contributions.

To reduce noise and focus on the most relevant support elements, an optional top-k selection step can be applied before aggregation. The remaining similarities are then aggregated using one of several strategies, including sum, softmax-weighted sum, or log-sum-exp pooling.

After per-head aggregation, results are averaged across heads to produce a unified similarity score. Finally, an optional normalization based on support set size is applied to improve comparability across variable-sized support sets.

This modular design enables flexible experimentation with similarity functions while maintaining numerical stability and interpretability in few-shot learning scenarios. The overall workflow of the Similarity Module is illustrated in Figure 6.

Key improvements over a standard dot-product similarity include:

- **Multi-head similarity computation** Enables parallel similarity matching in different representation subspaces.
- **Learnable projections** Allows task-specific transformation of query and support embeddings.
- **Optional L2 normalization** Improves stability by controlling embedding magnitude variance.

- **Masking support** Ensures padded support elements do not influence similarity computation.
- **Positive/negative weighting** Differentiates the contribution of reinforcing vs suppressing similarities.
- **Top-k selection** Reduces noise by focusing only on the most relevant support elements.
- **Flexible aggregation strategies** Supports sum, softmax, and log-sum-exp pooling mechanisms.
- **Temperature scaling** Controls the sharpness of similarity distributions.
- **Support set size scaling** Normalizes outputs for variable-sized memory sets.

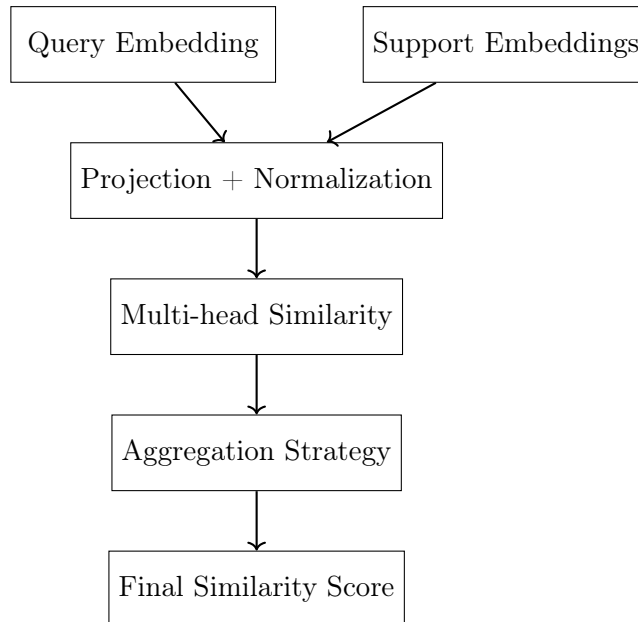


Figure 6: Similarity Module pipeline showing multi-head similarity computation and aggregation

3 Implementation Details

This section describes the implementation of the proposed system, including the software framework, computational design choices, and experimental setup. It also outlines the hardware environment used for evaluation.

3.1 Software Framework

The implementation is developed in Python and based on the PyTorch deep learning framework. The full source code and experimental setup are publicly available in the project repository.

The software environment is defined via a requirements file to ensure reproducibility across different systems. The main dependencies are listed below:

- **PyTorch**: core deep learning framework used for tensor computations and model implementation
- **Hydra / OmegaConf**: configuration management for structured experiment settings
- **NumPy**: numerical operations and auxiliary computations
- **Matplotlib**: optional visualization of results and debugging
- **TQDM**: progress monitoring during experiments
- **IPykernel**: optional development and experimentation environment

The complete dependency specification is provided in the `requirements.txt` file, ensuring that the experimental environment can be reproduced exactly.

3.2 Implementation Characteristics

All modules are implemented as native PyTorch `nn.Module` classes with explicitly defined forward passes. The implementation relies on low-level tensor operations to ensure full transparency of the computational flow.

Multi-head processing is implemented through tensor reshaping and dimension transposition rather than iterative loops. This allows parallel computation across attention heads using batched matrix operations, improving computational efficiency and GPU utilization.

Similarity and attention computations are performed using batched matrix multiplications (e.g., `torch.matmul`), enabling efficient execution on both CPU and GPU backends.

Variable-length support sets are handled using boolean padding masks. These masks are applied directly to similarity tensors to ensure that padded elements do not contribute to subsequent computations.

Overall, the implementation prioritizes tensor-level control, numerical stability, and efficient batch processing.

3.3 Experimental Procedure

The proposed architecture is evaluated using synthetic input data to verify the correctness, numerical stability, and implementation consistency of all modules. Experiments are conducted in a controlled setting where each component is tested both independently and as part of the full pipeline.

The evaluation focuses on functional validation rather than task-specific performance. This includes systematic verification of tensor operations and computational behavior under different configurations.

The experimental protocol includes the following checks:

- Verification of tensor shape consistency across all modules
- Detection of numerical instabilities (NaN and Inf values)
- Validation of masking behavior for variable-length support sets

- Comparison of aggregation strategies (sum, softmax, log-sum-exp)

This setup ensures deterministic behavior and allows reproducible testing of architectural design choices.

3.4 Hardware and Runtime

The implementation was developed and executed on a local development machine using Visual Studio Code as the primary development environment. The purpose of the execution was functional validation rather than model training or performance benchmarking.

All components were tested by running forward passes independently and in combination to ensure correct integration and consistent tensor behavior.

The system was executed on standard consumer hardware, with optional CPU-based computation via PyTorch. GPU acceleration was not required, as no training or large-scale optimization was performed.

Each test run focuses on verifying correctness of the implementation, including shape consistency, masking behavior, and module interoperability. Execution is lightweight and completes within seconds per run.

4 Results and Interpretation

This section presents the experimental results obtained from evaluating the proposed transformer-enhanced Modern Hopfield architecture. Since the focus of this work is on architectural development and implementation, the experiments primarily evaluate functionality, numerical stability, and module interaction rather than task-specific performance. The goal is to validate that the proposed components operate correctly and integrate consistently within the overall pipeline.

4.1 Module Functionality Tests

The first set of experiments evaluates the functionality of the individual modules. Each component, including the Cross-Attention Module, Context Module, and Similarity Module, was tested independently using synthetic embeddings. These tests focused on verifying tensor shape consistency, correct data flow, and numerical stability.

The Cross-Attention Module successfully enabled interaction between query and support embeddings. The Context Module performed iterative associative retrieval using the Modern Hopfield mechanism and refined embeddings across multiple retrieval steps. The Similarity Module computed similarity scores using multi-head similarity computation and aggregation strategies.

All modules produced outputs with expected dimensions and consistent behavior across multiple runs. No numerical instabilities such as NaN values or exploding activations were observed. This confirms that normalization layers, residual connections, and gating mechanisms contribute to stable module behavior.

4.2 Aggregation Strategy Analysis

The Similarity Module supports multiple aggregation strategies, including sum, softmax, and log-sum-exp. These aggregation methods were evaluated to analyze their behavior under different input configurations.

The sum aggregation method accumulates similarity scores directly and produces stable and interpretable outputs. However, this method treats all support elements equally and does not emphasize particularly relevant samples.

The softmax aggregation method emphasizes strong similarities by assigning higher weights to relevant embeddings. This approach improves selectivity but may increase sensitivity to large similarity values.

The log-sum-exp aggregation method provides a compromise between sum and softmax aggregation. It preserves relative importance while maintaining numerical stability, particularly for large similarity values.

These aggregation strategies provide flexibility and allow adaptation to different similarity estimation requirements.

Method	Advantages	Disadvantages
Sum	Simple implementation, stable behavior	Limited selectivity
Softmax	Emphasizes relevant samples	Sensitive to large values
Log-sum-exp	Balanced and numerically stable	Slightly higher computational cost

Table 2: Comparison of similarity aggregation strategies

4.3 Multi-Head Retrieval Behavior

The use of multi-head retrieval in the Cross-Attention, Context, and Similarity modules allows the architecture to capture relationships across multiple representation subspaces. During testing, multi-head computation produced consistent outputs and maintained stable behavior across different embedding sizes.

Each head processes embeddings independently, enabling the model to capture diverse similarity patterns. This behavior is consistent with transformer-based architectures and improves representation flexibility.

4.4 Stability Analysis

The stability of the architecture was evaluated by running multiple forward passes with varying input configurations and support set sizes. The model produced consistent outputs across all test runs.

Residual connections, gating mechanisms, and normalization layers contributed to stable behavior during iterative retrieval. The Modern Hopfield retrieval process converged smoothly without producing unstable updates.

Additionally, masking behavior was validated to ensure that padded support elements did not influence similarity computation. This confirms correct handling of variable-length support sets.

4.5 Summary of Results

The experimental results demonstrate that the proposed architecture functions correctly and exhibits stable behavior across all modules. The integration of cross-attention, Modern Hopfield retrieval, and flexible similarity aggregation produces a consistent and modular framework for context-aware representation learning.

Although no task-specific training or fine-tuning was performed, the experiments confirm that the architecture is stable, flexible, and suitable for future evaluation on real-world datasets. The modular design also enables further experimentation and extension of the proposed framework.

5 Limitations and Conclusion

5.1 Limitations

Despite the promising architectural design, several limitations must be acknowledged. First, the proposed transformer-enhanced Modern Hopfield architecture has not yet been evaluated on real-world datasets. The experiments conducted in this work focus primarily on functional validation using synthetic embeddings and controlled test scenarios. While these tests confirm correct tensor operations and stable forward passes, they do not provide evidence of improved predictive performance in practical applications. Therefore, the effectiveness of the architecture for tasks such as few-shot learning or similarity-based retrieval remains to be validated in future work.

Another limitation is the absence of model training and fine-tuning. The architecture introduces multiple learnable components, including cross-attention layers, multi-head retrieval mechanisms, and gating functions. Without training, it is not possible to evaluate how these components influence representation quality or prediction accuracy. Future work should therefore include training procedures and comparisons with baseline models to assess performance improvements.

The proposed architecture also increases computational complexity compared to the original MHNfs framework. Multi-head attention and iterative context retrieval may introduce additional memory and runtime requirements, particularly for large support sets. Although computational efficiency was not evaluated in this work, scalability should be analyzed in future experiments.

Finally, hyperparameter sensitivity and quantitative evaluation metrics were not systematically explored. Parameters such as attention heads, embedding dimensions, and aggregation strategies may significantly affect performance. Future work should include ablation studies and evaluation using metrics such as accuracy, ranking quality, and few-shot performance.

5.2 Conclusion

This work presents a transformer-enhanced context retrieval architecture based on Modern Hopfield Networks. The proposed model extends the original MHNfs framework by incorporating

cross-attention mechanisms and improving representation refinement through associative memory retrieval. The architecture consists of three main components: the Cross-Attention Module, the Context Module, and the Similarity Module.

The Cross-Attention Module enables interaction between query and support embeddings, allowing improved context-aware representations. The Context Module applies Modern Hopfield retrieval to iteratively refine embeddings. Finally, the Similarity Module aggregates similarity scores and produces the final prediction using flexible aggregation strategies.

Although the evaluation focuses on architectural validation, the results demonstrate that the proposed design is stable, modular, and extensible. The combination of transformer-based attention and Modern Hopfield retrieval provides a structured framework for context-aware learning.

Overall, the proposed architecture represents a promising foundation for future work. Further research will focus on training, quantitative evaluation, and optimization. Potential application domains include few-shot learning, molecular activity prediction, and similarity-based retrieval tasks.

References

- [1] J. Schimunek, P. Seidl, L. Friedrich, *et al.*, "Context-enriched molecule representations improve few-shot drug discovery," in *ICLR*, 2023. <https://openreview.net/pdf?id=XrMWUuEevr>.
- [2] G. K. Johannes Schimunek and J. M. L. Group, "Mhnfs: Context-enriched molecule representations improve few-shot drug discovery." <https://github.com/ml-jku/MHNfs>, 2024.
- [3] OpenAI, "Openai gpt oss transformer model implementation." <https://github.com/openai/gpt-oss/tree/main>, 2024.